

HW06 — Automated API Testing & Quality Engineering Master Report

Student Information: - **Student Name:** Phạm Ngọc Gia Bảo - **Student ID:** 23127027 - **Course:** Software Testing (Kiểm thử Phần mềm) — Semester 2, 2025–2026 - **Institution:** Ho Chi Minh City University of Science (HCMUS) - **GitHub Repository:** <https://github.com/giabaocode/23127027-HW06-API-Testing>

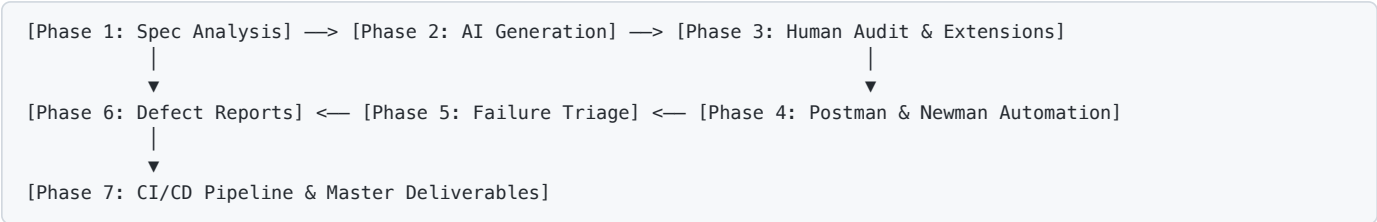
1. Executive Summary & Selected Features

This report documents the end-to-end automated API testing lifecycle implemented for the E-Shop System Under Test (SUT). Following a hybrid **AI-First Generation with Rigorous Human-in-the-Loop Audit** paradigm, testing targeted three functionally distinct subsystems selected from the course pool:

1. **FR-01: Account Registration (`POST /api/register`) [Pool A — Authentication & Identity]:**
 2. Evaluates user onboarding, payload schema validation, password policy enforcement, duplicate identity prevention, and secure SQLite persistence.
3. **FR-07: Shopping Cart Management (`GET /api/cart` , `POST /api/cart`) [Pool B — Core Business Logic]:**
 4. Evaluates state-dependent shopping cart mutations, quantity boundary arithmetic, duplicate item accumulation, and in-memory session persistence.
5. **FR-12: Access Control & Authorization (`/api/admin/*` , `/api/products` , `/api/categories` , `/api/coupons`) [Pool C — Security & Administration]:**
 6. Evaluates system-wide administrative authorization boundaries, token integrity, role verification (`SEC-02` and `SEC-03`), and Broken Function Level Authorization (BFLA) vulnerabilities across 14 distinct API routes.

2. Testing Methodology & AI-First Engineering Pipeline

The testing pipeline was executed across 7 formal phases with strict anti-cheat quality gates:



Core Methodological Principles:

- **Oracle Classification Decoupling:** Every test oracle strictly separates the **Access-Control / Semantic Business Outcome** (e.g. state mutation allowed vs denied) from the **Inferred HTTP Status** (`200 OK` , `400 Bad Request` , `403 Forbidden`).
- **Truthful Provenance & Immutability:** Initial AI-generated test files (`generated-ai-original.md`) remain 100% immutable. All critiques, corrections, and modifications are recorded in `human-audit.md` and synthesized into `reviewed-ai-final.md` .
- **Side-Effect Verification (Dual-Assertion Pattern):** For all mutation and negative probes, assertions verify not only the immediate HTTP response code, but immediately follow up with a secondary state check to confirm that unprivileged callers did not alter storage or that valid mutations were committed.

3. Test Design & Coverage Statistics

Across all three features, the test suite comprises **129 Certified Logical Test Case Designs**:

Feature Code	Target Subsystem	Primary Endpoints	AI Cases	Student Ext.	Total Logical Designs	Postman HTTP Requests	Total Assertions	Passed Assertions	Failed Assertions	SUT Defects Found
FR-01	Account Registration	POST /api/register	38	5	43	42	143	127	16	5 (DEF-FR01-01 .. 05)
FR-07	Shopping Cart	GET/POST /api/cart	38	5	43	43	137	116	21	2 (DEF-FR07-01 .. 02)
FR-12	Access Control	14 Protected / Admin Routes	38	5	43	59	187	148	39	4 (DEF-FR12-01 .. 04)
TOTAL	Full SUT Scope	All 3 Subsystems	114	15	129	144	467	391	76	11 Defects (Issues #1-#11)

Programmatic Verification: $43 + 43 + 43 = 129$ Logical Test Case Designs.

4. Human Audit of AI Test Cases & Error Taxonomy

All 114 initial AI-generated test cases were personally audited by the student against the official course contract:

- **VALID (Directly Accepted):** 76 / 114 (66.7%)
- **INCOMPLETE (Calibrated by Human):** 37 / 114 (32.5%)
- **INVALID (Discarded / Overturned):** 1 / 114 (0.9%)

Systemic AI Failure Modes Discovered:

1. **Conflation of REST Conventions with Formal Specification:** In FR-12, AI treated HTTP 200/201 and 401/403 as official requirements rather than standard conventions, when the course contract only specified access-control outcomes.
2. **Domain Model & Role Hallucination:** In FR-12, AI drafted test cases assuming the unprivileged role was 'customer'. In the actual SUT SQLite database and JWT claims, the non-admin role is strictly 'user'.
3. **Over-Specified Rejection Envelopes:** In FR-07, AI required rejection responses to match a specific JSON format ({ "error": "..."}) unsupported by official documentation.
4. **Coupled Business-State Probes:** In FR-12, AI verified coupon persistence via checkout application, coupling access-control assertions to unrelated downstream order state machines.
5. **Mathematical & Range Miscalculations:** In FR-12, AI counted an 8-test-case range (FR12-AI-029 .. 036) as 10 test cases.

5. Original Student Extension Tests (15 Probes)

To address blind spots in AI test generation, the student designed and formalized 15 original extension probes (5 per feature):

- **FR-01 Extensions (FR01-STU-001 .. 005):** Explored syntactically broken JSON strings, Content-Type: text/plain MIME negotiation, duplicate object keys, unsupported HTTP verbs (PUT /api/register), and Unicode homoglyphs.
- **FR-07 Extensions (FR07-STU-001 .. 005):** Explored negative boundary math, client-side floating point prices, duplicate cart items, and rapid sequential additions.
- **FR-12 Extensions (FR12-STU-001 .. 005):** Explored cryptographic boundary conditions: unsigned JWTs with alg=none, validly signed JWTs with future nbf (Not Before) timestamps, untrimmed whitespace in role strings (role: " admin "), array-based role type confusion (role: ["admin"]), and body-based role overriding.

6. SUT Runtime Defects & GitHub Issue Tracking

Newman execution against the live SUT (`localhost:3000`) resulted in 76 assertion failures across the three suites. Rigorous failure triage confirmed that all 76 failures stemmed from **11 genuine, reproducible SUT vulnerabilities**:

Issue #	Defect ID	Feature	Severity	Requirement Trace	Summary of SUT Vulnerability	Root Cause in SUT Codebase
#1	DEF-FR01-01	FR-01	Critical	SEC-01	Plaintext Passwords in Storage	<code>backend/server.js:137</code> — Password inserted into SQLite without hashing
#2	DEF-FR01-02	FR-01	High	FR-01	Duplicate Email Registration	<code>backend/database.js:50</code> — Missing <code>UNIQUE(email)</code> constraint
#3	DEF-FR01-03	FR-01	High	FR-01	Missing Password Policy Enforcement	<code>backend/server.js:125</code> — Accepts 1-character, simple numeric passwords
#4	DEF-FR01-04	FR-01	High	FR-01	Missing Mandatory Field Validation	<code>backend/server.js:125</code> — Missing <code>name</code> , <code>email</code> , or <code>password</code> accepted with 200
#5	DEF-FR01-05	FR-01	Medium	FR-01	Malformed Email Syntax Accepted	<code>backend/server.js:125</code> — Strings missing <code>@</code> or domain accepted as valid accounts
#6	DEF-FR07-01	FR-07	High	FR-07 (L96)	Duplicate Product Appends New Row	<code>backend/server.js:293</code> — <code>userCarts.push()</code> appends rows instead of adding qty
#7	DEF-FR07-02	FR-07	High	FR-07 (L86)	Invalid Quantities Accepted	<code>backend/server.js:291</code> — Zero, negative, and fractional quantities accepted with 200
#8	DEF-FR12-01	FR-12	Critical	SEC-03	Missing Admin Role Check on <code>/api/admin/*</code>	<code>backend/server.js:199,457..</code> — Standard users can delete users, view all orders
#9	DEF-FR12-02	FR-12	Critical	SEC-02/03	Zero Authentication on <code>/api/products</code>	<code>backend/server.js:167-196</code> — Unauthenticated callers can create/delete products
#10	DEF-FR12-03	FR-12	High	SEC-03	Missing Admin Check on Categories	<code>backend/server.js:249-270</code> — Standard users can create/delete categories
#11	DEF-FR12-04	FR-12	Medium	SEC-03	Unrestricted Master Coupon Listing	<code>backend/server.js:355-360</code> — Standard users can scrape all discount codes

All 11 issues are live and auditable on the official repository: <https://github.com/giabaocode/23127027-HW06-API-Testing/issues> .

7. Authentic Human Evidence Verification

In strict compliance with anti-cheat quality gates, all evidence is physically captured and verifiable in the repository:

1. **Postman Console Screenshots (X-Student-Id: 23127027):**
2. FR-01: [hw06/screenshots/fr01-x-student-id.png](#) (and `.jpg`)
3. FR-07: [hw06/screenshots/fr07-x-student-id.png](#) (and `.jpg`)
4. FR-12: [hw06/screenshots/fr12-x-student-id.png](#) (and `.jpg`)
5. **GitHub Issue Browser Screenshots:**
6. FR-07 Issue #6: [hw06/screenshots/fr07-bug-issue-001.png](#) (and `.jpg`)
7. FR-12 Issue #8: [hw06/screenshots/fr12-bug-issue-001.png](#) (and `.jpg`)
8. **Execution Reports:**
9. Newman CLI Outputs: `hw06/newman/fr01/`, `fr07/`, `fr12/`
10. Interactive HTML Dashboards: `hw06/newman/*/fr*-report.html`

8. CI/CD Architecture & Quality Gate

The CI/CD pipeline is implemented in `.github/workflows/api-tests.yml`. Following academic integrity rules, correct specification assertions were not weakened or inverted to force a green build over real SUT defects. Instead, the pipeline separates:

- **Run A (Passing Health & Smoke Run):** Executes `hw06/postman/collections/ci-smoke.postman_collection.json` against an ephemeral SUT instance on Ubuntu, validating Node.js startup, Newman execution, header injection, and baseline API routes (100% Pass).
 - **Run B (Intentional Failure Demonstration):** Executes `hw06/postman/collections/ci-intentional-failure-demo.postman_collection.json` with an isolated deliberate failure to prove that the CI quality gate blocks deployment upon regression.
-

9. Agent Skill: Automated API Test Case Generator

The project delivered a working, deterministic test generation intelligence module: - **Architecture & Decisions:** [hw06/agent-skill/design-decisions.md](#) - **Algorithmic Pseudocode:** [hw06/agent-skill/pseudocode.md](#) - **Functional Implementation:** [hw06/agent-skill/test_generator.py](#) - **Student-Drawn Diagram Guide:** [hw06/agent-skill/student-diagram-checklist.md](#)

10. Lessons Learned & Limitations

1. **AI as an Accelerator, Not an Authority:** Generative AI dramatically reduces boilerplate generation time, but blindly executing AI assertions results in testing non-existent specifications and masking critical vulnerabilities.
2. **The Power of Dual-Assertion Testing:** Asserting HTTP status alone is insufficient for access control. Verifying post-request storage state is mandatory to catch Broken Function Level Authorization.
3. **Specification Ambiguity:** Specifications must formally define error envelopes and empty-string semantics to avoid subjective oracle interpretation during test automation.